

AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve quality, safety, and cost efficiency of AI agent, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. To answer similar questions in traditional systems software, profiling attributes resource consumption to responsible entities, such as code paths, by aggregation. Yet existing agent observability tools are mainly for single-execution debugging or tracing instead of profiling, which makes it difficult to answer the above questions when the trajectories are becoming long and complex. Agent behavior differs from code execution, creating two challenges for profiling: the responsible entities are semantic categories such as task intent and workflow phase, not code paths, and agent trajectories lack the stable identifiers and structural hierarchies that make attribution possible. We propose a *semantic operation stack model* that adapts profiling to agent trajectories. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack, enabling hierarchical attribution at different levels of granularity from the same data. AGENTPROF implements this model as a profiler with pluggable algorithms for *intent attribution* and *stack construction*, compiling local agent trajectories into pprof-compatible profiles. On real trajectories and public datasets, our evaluation shows that semantic profiling effectively attributes cost and locates problems across agent trajectories.

Introduction

AI agents (Yang et al. 2024; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours and days, with a large number of interactions each, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume

the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Answering these questions requires analysis and evaluation across many trajectories (Mazaheri and Mazaheri 2026; Lù et al. 2025), a task becoming costly for manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023) at scale. In traditional systems software, profiling answers these questions by attributing resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing.

Yet existing agent tools support debugging and tracing but not profiling (§). Some (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) organize events into per-execution span trees, effective for diagnosing a single run but unable to aggregate by semantic category. Others (DataDog 2025; Laminar 2025) cluster inputs or extract structured signals, but characterize *input distributions* (“30% of users ask code questions”) rather than *resource attribution* (“review tasks consume 40% of the token budget”), because tags at the request level do not propagate to downstream tool calls and system effects.

Adapting profiling to agent trajectories is challenging (§). In traditional software, the responsible entities are code paths. Profiling is straightforward because function names are stable and runtime call nesting provides the attribution hierarchy (§). Agent behavior differs fundamentally. To answer the questions above, the responsible entities need to be semantic categories such as task intent and tool operation, not code paths. Agent trajectories lack the two properties that make traditional profiling possible. Prompts are natural language, so two prompts expressing the same intent share no common string. Agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames: a sequence of prompts can be a task or multiple tasks, a task can be part of a bigger task.

Despite these differences, the fundamental profiling methodology transfers to agent trajectories: project resources onto responsible entities, assign stable tags, and attribute hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* pro-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

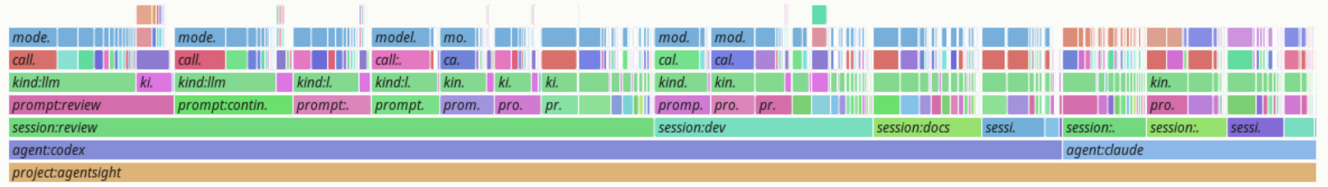
agentpprof tokens profile

prefix-merged flamegraph; width = count; total = 21899030768; drawn nodes = 940; hidden tiny nodes = 7331; depth = 7



agentpprof time profile

prefix-merged flamegraph; width = seconds; total = 2263796; drawn nodes = 865; hidden tiny nodes = 7474; depth = 10



agentpprof files profile

prefix-merged flamegraph; width = count; total = 46009; drawn nodes = 2051; hidden tiny nodes = 19126; depth = 7

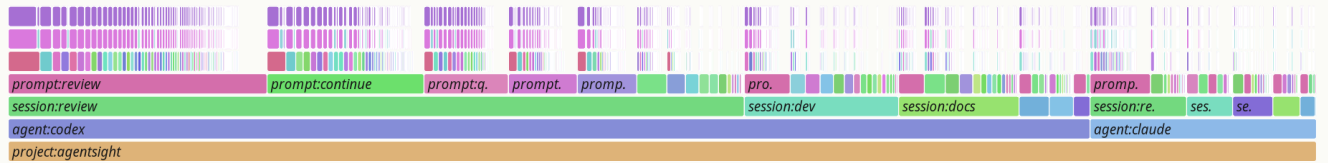


Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. *Top (tokens)*: width represents token count. *Middle (time)*: width represents wall-clock duration. *Bottom (files)*: width represents file-operation count. All three are produced from the same operations by changing only the stack fields and weight function.

vide hierarchical attribution, replacing the runtime call stack. Choosing different fields changes the attribution granularity: the same operations can be attributed by task category, by execution phase, by session, or by action type (Figure 1).

We implement this model in **AGENTPROF**, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Figure 2). Two pluggable algorithm frameworks enrich the model. *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This gives agents the stable tags that traditional profiling gets for free from function names. *Stack construction* builds the attribution hierarchy, either from a user-specified field list or by automatically detecting boundaries between distinct work phases. On 325 real Codex and Claude trajectories (183,714 system observations), semantic profiling separates over 90% of system-effect cost that per-session views cannot attribute to task categories. On four public datasets (34,539 operations) where the correct annotations are known but hidden from the profiler, ranked groups locate real problems at 9.4% inspection work, with 45% fewer groups than per-session grouping. Derived tags agree with ground truth on 7 of 9

held-out datasets.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (§–§).
2. **System: AGENTPROF.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output (§).
3. **Evaluation.** We evaluate profiler output against hidden ground-truth trajectory annotations, analogous to injecting known bottlenecks in CPU profiler evaluation, on 325 real trajectories and 4 public datasets (§).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers system effects at the lower layer: process spawns, file reads and writes, and

network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

System Profiling

Profiling attributes resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing. In traditional software, the profiling pipeline works as follows: a profiler periodically samples execution state, attaches each sample to the current call stack, and merges samples with identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize the result, where width or node size represents aggregate cost. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools support flexible aggregation, but they all assume stable function names and a runtime call stack. Pprof’s `tagroot/tagleaf` options can add label-derived pseudo-frames, but only on top of an existing execution stack that agent trajectories do not have.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees, effective for debugging a single run. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing tools (OpenTelemetry 2024; Arize AI 2024b) do not capture these categories because prompts are natural language: two prompts and tool invocations expressing the same intent may be different, so the profiler cannot aggregate them the way it aggregates identical function names. The second challenge is that agent trajectories have no runtime hierarchy for attribution. In CPU profiling, the call stack determines attribution at every level: `malloc`’s cost belongs to `parse`, to `process`, and to `main` simultaneously. A sequence of prompts may constitute one task or several, and a task may be part of a larger workflow, but no runtime mechanism marks these boundaries or determines at which granularity to attribute cost. We quantify this in RQ1 (§).

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

R1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span two layers: intent (prompts, LLM calls) and system effects (file I/O, process spawns). The profiler must connect system-layer resource consumption to intent-layer semantic categories.

R2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

R3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy: function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent events are not nested by execution, so the profiler must construct attribution hierarchies from the data.

Design

The three requirements (cross-layer resource projection, stable tags, and hierarchical attribution) drive the design. Operations (§) satisfy R1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Intent attribution* (§) satisfies R2 by deriving stable tags from natural-language fields. *Operation stacks* (§) satisfy R3 by replacing the runtime call stack with query-time field projections that attribute resources at multiple levels of granularity, so the same data supports multiple views without rebuilding the input. The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in §, a profiler should not distinguish between them in the data representation. AGENT-PROF therefore represents every activity as a single abstraction, the operation, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values.

An operation stack provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role: an ordered list of fields $[f_1, f_2, \dots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \dots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore be attributed as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory using the dataset’s own annotations, and changing the fields changes the attribution without changing the data. Sessions and spans are optional operation fields, not fixed hierarchy levels, so the same data supports both debugging views (include `session`) and

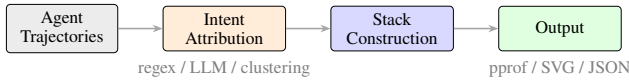


Figure 2: Data-flow and algorithm pipeline. Local agent histories enter through `agent-session` parsing; public datasets enter as operation JSONL. Both paths produce uniform operations. Intent attribution, stack construction, and folding are query-time algorithms applied identically to both sources.

aggregate profiling views (exclude it). A view is a triple (φ, σ, w) : a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \dots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Four built-in views cover common questions: `tokens` (LLM calls weighted by token count), `time` (all timed operations weighted by duration), `files` (file operations weighted by event count), and `network` (network operations weighted by event count).

Recursive Operation Segmentation

The algorithm rests on one structural intuition: an agent’s task occupies a contiguous span of the trajectory, and it decomposes into smaller contiguous subtasks. Task structure is therefore exactly a set of nested named intervals, and recovering it only requires finding the transition points between them.

The core algorithm is segmentation, not labeling: instead of assigning each step a class from a predefined taxonomy, the backend reads a trajectory and finds the transition points where the agent’s current responsibility changes. At each transition it cuts the trajectory and names the intervals on both sides; it then recurses into each interval, cutting again wherever a finer responsibility change remains, and stops when an interval reads as one coherent piece of work. Formally, a trajectory is an ordered turn sequence $T = (t_1, \dots, t_n)$, and a segmentation is a set S of named intervals over T that is nested (any two intervals are disjoint or one contains the other), covers every turn, and contains the whole-session interval as its root. The backend computes S by one recursive rule: `SEGMENT( $I$ )` selects zero or more transition points inside interval I , which partition I into consecutive child intervals; each child receives a short action-first name and is segmented recursively; selecting no transition terminates the branch and declares I one operation. A turn’s operation path is the name chain of the intervals containing it, from the session root to its innermost interval; equal paths fold across sessions. Segmentation is the right primitive for three reasons. First, transitions are what a trajectory actually exhibits: agent work has no predefined taxonomy, but a shift in what the agent is trying to do is visible in the source content itself. Second, cuts compose: nested intervals form a hierarchy whose depth emerges per branch from the content rather than from a prescribed schema. Third, naming intervals rather than steps yields identifiers at the granularity where recurrence lives, so the same short action-first name (one to three words) reappears across sessions and folds. Any backend able to find

transitions and name intervals—an agent, a plain language model, or a statistical rule—implements the same contract.

Implementation

AGENTPROF is an offline, post-hoc profiler implemented as a Rust CLI ($\sim 9.8\text{K}$ LOC, Figure 2). It reads Codex and Claude Code local JSONL history files and AgentSight (Zheng et al. 2025) recordings for system effects, and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. The backend works in a three-file annotation workspace (`trace.jsonl`, `annotation.json`, `stacks.folded`); the CLI validates that marks are nested, noncrossing, and cover every source node, then emits one standard pprof protobuf profile that existing tools such as `go tool pprof` read directly. Stack emission is deterministic: each source node contributes the agent, its covering semantic operation path, and its retained LLM-call and tool-call evidence, with raw session and prompt identities kept as pprof labels rather than visible frames, so equal semantic prefixes fold across sessions; switching the additive measure replays the same annotation and changes only stack widths.

Evaluation

We evaluate three data classes. *Real inputs*: 41 long-horizon coding-agent sessions, 440 mixed-outcome web-agent trajectories (Lù et al. 2025), and 42 development sessions from the authors’ workstation. *Annotated trajectories*: 405 Code-TraceBench trajectories with 20,866 operations and 2,948 human-verified stages (Li et al. 2026), 287 OSWorld-Human sessions (WukLab 2025), 1,012 AgentBoard goals (Ma et al. 2024), and 2,737 published action labels (Bouzenia and Pradel 2025). *Problem-localization benchmarks*: the complete AgentProcessBench, HINTBench, and TraceElephant workloads with independent targets over 27,346 operations (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026).

RQ1: Does Semantic Profiling Improve Resource Attribution?

The automatic backend assigns 96 recursive annotations to the 735 source nodes of three independent executions of one real Git-deployment task (489 operations, 4,558,192 provider-reported tokens, semantic depth five with no fixed limit; Figure 3). By operation count the repeated `diagnose authentication` subtree is 21.47% of the task; by tokens it is 46.15%, and source drilldown shows all three executions verifying substitute transport paths without establishing the requested password-authenticated endpoint. A same-input organization control replays the same 489 operations and weights through native, coarse-action, and semantic middle organizations: the SSH responsibility scatters across 105 native calls and six coarse action kinds (largest branch 39.42% of its tokens), while only the semantic organization yields one focusable cross-run responsibility. At population scale, replaying the frozen 440-session hierarchy (7,229 operations and 51,904,621 tokens, both conserved exactly) ranks the same operations once by count and once by tokens: mean

Workload	Direct+ AgentProf	Direct+Raw +Evidence	Direct only	AgentProf only
AgentProcessBench	.894	.893	.863	.791
HINTBench	.517	.518	.411	.432
TraceElephant	.326	.324	.209	.259

Table 1: Standard per-query AP averaged as MAP over the complete RQ2 populations. Higher is better.

per-task Kendall’s tau-b (Kendall 1938) is 0.886 (95% task-cluster interval [0.857, 0.915]) with Spearman’s rho (Spearman 1904) 0.935, and 10 of 77 rankable tasks fall below 0.7—dominant responsibilities are stable for most web-scale tasks, and the regimes where the measure changes the decision are exactly where multi-measure replay pays. Profiling the authors’ own 42 development sessions—the agents that built AGENTPROF—materializes 10,423 source nodes carrying 1,380,863,014 token components; one fixed-instruction pass yields 1,737 annotations with zero backend failures, the largest token path holds only 1.735% of mass, and the three longest sessions carry distinct dominant responsibilities.

RQ2: Does Profiler Output Correspond to Real Problems?

On the three complete localization workloads (614, 400, and 220 target-bearing queries), operations are ranked lexicographically by the benchmark-native direct diagnostic and a target-blind Agent+Evidence group score; we report non-interpolated per-query AP averaged as MAP (Robertson 2008) (Table 1).

Direct+AgentProf raises MAP over Direct-only by .031 [.024, .039], .107 [.093, .120], and .117 [.088, .148] under 10,000 paired trajectory-cluster resamples; an information-matched raw-action refinement ties, and the tie follows from the mechanism: per-operation anomaly signal resides in the retained source evidence that both views share. Used as a reading index on TraceElephant, the semantic skeleton guides a strong external reader to ranking quality statistically equal to an information-matched raw-action skeleton while opening 53.0% versus 65.0% of the source evidence (paired delta +.120 [+ .103, +.137]); full-trace reading reaches MAP .502 versus .209 and .326 at 12,615 input tokens per query, but is bounded by the context window, which populations like the 4,558,192-token Git task exceed. Across all 338 bad-good pairs of the mixed-outcome AgentRewardBench population, a target-blind signed profile places recovery at 44.6% of failed-side versus 12.0% of successful-side occurrences (completion 1.8% versus 5.1%), and per-trajectory recovery exposure predicts independent expert looping labels at AP .634 against prevalence .398 (interval [.181, .293]).

RQ3: How Accurate Are the Tags?

Direct Agent annotation on all 405 CodeTraceBench trajectories reaches 0.764 ordinary B^3 F1 (Bagga and Baldwin 1998) and 0.480 exact adjacent-boundary F1 (Ruokolainen et al. 2016) against the 2,948 human stages, versus 0.663 and 0.266 for multi-resolution recurrence (paired deltas +0.101

Method	B^3 P	B^3 R	B^3 F1	Bound. F1
Direct Agent annotation	0.793	0.736	0.764	0.480
Multi-resolution recurrence	0.782	0.575	0.663	0.266
Native source tree	0.975	0.249	0.397	0.259
Raw-action grouping	0.891	0.388	0.541	—

Table 2: Automatic operation-structure agreement on all 405 CodeTraceBench trajectories.

[0.087, 0.116] and +0.214 [0.193, 0.235]) and 0.541 B^3 F1 for raw-action grouping (Table 2); the marks conserve exactly 20,866 operations and 494,862,929 provider-reported tokens.

On 287 OSWorld-Human sessions, a supervised boundary predictor (McCallum and Nigam 1998) reaches 0.739 boundary F1 and 0.816 B^3 F1 and label-free recurrence reaches 0.680 and 0.786, versus 0.645 and 0.678 for the strongest control; closed-label task-family and action backends reach 0.695 and 0.498 macro-F1 (Lewis et al. 2004; Ma et al. 2024; Bouzenia and Pradel 2025) against majority baselines of 0.044 and 0.061.

RQ4: What Is the Profiling Cost?

With marks fixed, the current binary constructs a 27,765-operation union profile in 1.16 s at 465.2 MiB peak RSS (0.0418 ms per operation, $R^2 = 0.9997$), 19.6% more time than raw-action grouping. Direct annotation of all 405 CodeTraceBench trajectories consumes 12,050,384 input and 231,886 output tokens in 2,215.9 s of active backend wall time, and the full deterministic downstream pipeline takes 11.5 s; annotating the complete 440-session AgentRewardBench population takes 58.7 minutes and 12,039,417 input tokens (90.8% cached), while materialization stays sub-second, so construction cost is dominated by the one-time automatic backend and every later measure switch is a cheap replay.

Related Work

Agent observability tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026c) focus on single-execution debugging. Datadog LLM Observability (Datadog 2025) and Laminar (Laminar 2025) cluster inputs or extract signals but characterize input distributions, not resource attribution. AgentRx (Barke et al. 2026), TEL-Bench (Wang et al. 2026a), AgentAtlas (Mazaheri and Mazaheri 2026), TrajAD (Liu et al. 2026), and AgentFixer (Mulian et al. 2026) perform per-trajectory fault localization. AGENTPROF complements all of these with category-level aggregate profiling across trajectories.

Conclusion

Agent observability needs profiling, not only debugging. AGENTPROF shows that the semantic operation stack model, with pluggable intent attribution and stack construction, brings hierarchical attribution to agent trajectories. Scoring profiler output against hidden trajectory annotations confirms



Figure 3: All three real `git-multibranch` executions under one fixed operation hierarchy. Width is operation count (top) or provider-reported tokens (middle); the bottom panel focuses the shared `diagnose authentication` operation. AGENTPROF emits only standard pprof; the renderings read the same `.pb.gz` through `go tool pprof`.

that semantic profiling separates over 90% of system-effect cost that per-session views leave mixed, localizes annotated problems at 9.4% inspection work versus flat summaries, and produces tags that agree with ground truth on 7/9 held-out datasets. Per-session grouping remains faster to a first positive on 4/6 tasks, confirming that profilers and debuggers are complementary, and AGENTPROF supports both by including or excluding session fields in the stack query. Multi-project evaluation and tracing-ecosystem integration are the most impactful next steps.

References

- Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- Arize AI. 2024a. Arize Phoenix. <https://arize.com/docs/phoenix>.
- Arize AI. 2024b. OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85.
- Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.
- Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857.
- Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.
- Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
- Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.
- Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y.

2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.
- Google. 2024a. Perfetto Trace Processor. <https://perfetto.dev/docs/analysis/trace-processor>.
- Google. 2024b. pprof. <https://github.com/google/pprof>.
- Gregg, B. 2011. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- Kendall, M. G. 1938. A New Measure of Rank Correlation. *Biometrika*, 30(1/2): 81–93.
- Laminar. 2025. Signals: Structured Event Extraction from Traces. <https://docs.lmn.ai/signals/introduction>.
- LangChain. 2024. LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- Langfuse. 2024. Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.
- Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.
- Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942.
- Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362.
- Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.
- McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.
- Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. In *Proceedings of the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE)*.
- OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR conference on Research and Development in Information Retrieval*, 689–690.
- Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.
- Spearman, C. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*, 15(1): 72–101.
- Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.
- Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.
- Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026c. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.
- WukLab. 2025. OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shi, D.; Tong, J.; Lu, K.-W.; Garg, V.; Wang, Y.; Dai, Z.; Li, F.; Bisk, Y.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSPP)*.